



Musica
Version V 1.1

Notes
Version 1.0

Table of Contents

Introduction

Using Musica

Advanced commands

Troubleshooting

Appendix A - SQL Database User Examples

Appendix B - Detailed Commands and Usage

Appendix C: List of Application and SQL sequences

Introduction

Musica is a cross-platform program that supports Linux, Windows 1(X) and MacOS. This SQL-based program uses python scripting to curate a users media content collection on their own local home computer without having to rely on other cloud-based DB platforms. Notes is a companion product that utilizes a browser-based interface over the command-line interface (CLI). Notes itself can be used as a personal tool or as a server-based application. A users basic information can be retrieved from Discogs or any other archival tool (i.e. spreadsheet, simple text file) as long as the information is a one record per line, semicolon formatted file. Users may have multiple copies of the same recording (i.e. 2 copies of an LP, an LP and a CD, a CD and Cassette or any combination for formats). A “Header Row” isn't required to prepare the test file for loading, although it is helpful for debugging failed imports. The database relies on a pre-loaded schema that dictates the columnar information in a file. If data is retrieved from Discogs, the user may need to make changes to the contents of the file prior to loading into the database. Because the program interacts with an SQL database, basic SQL commands are all that are required to interact with the dataset. Musica is a single-user database application. The database back-end is SQLite. This was chosen because of its small footprint, speed and robust code.

Before getting started, the user needs to be aware of a few conventions:

- Normal text is written in Times New Roman 12-pt font
- `System commands are written in Courier 10 pt`
- **Items in Bold face are required or need to know.**
- Appendix A contains SQL database examples
- Appendix B contains Detailed Commands and Usage
- Appendix C lists various Applications and SQL sequences
- For many operations (installation, Administration, Musica setup, etc) the user must have access as sudo (“superuser do” in *nix-like systems) or Administrator privileges in a Windows environment.

Installation instructions:

Musica and Notes each have installation guides that are part of the pkg tarballs. It is suggested to read the installation guides, especially for Notes as it contains specific information for using Musica on a domain server (commercial) or a personal application.

New Features:

Besides implementing the Notes tool, there was a minor change to the data tables to Musica. It is a very ubiquitous change that enables another genre option when adding/querying the musica database, either from the command-line or from the Notes browser. It WILL require the user to rebase any existing musica.db if they are already a Musica V1.0 user. Please refer to the Musica V1.1 installation guide on the steps required to perform a rebase on the musica database.

The primary uses of Notes/Musica are as a central data location tool running on an Apache or similar web-server, or on a personal computer. If Notes is used on a domain server, then handheld devices, such as a smart-phone or tablet can also be used to access/query a local database.

Notes is not required to use Musica, but Musica is required to take advantage of accessing the database content. Notes/Musica are compatible with all mainstream browsers such as Firefox, Chrome, Chromium, Opera, Internet Explorer. Installation instructions for each product are described in the installation instructions for each application.

Previous versions of Musica utilized two types of data entry:

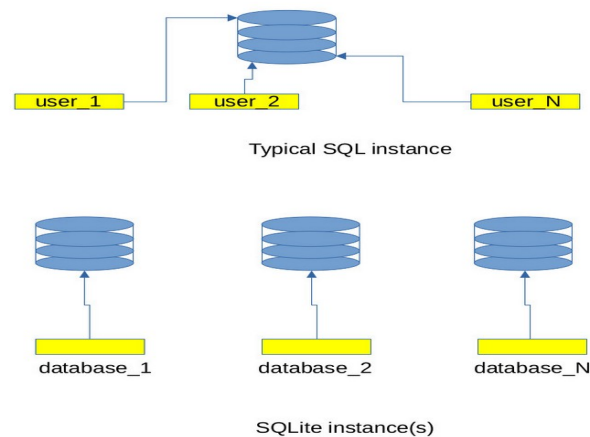
1. Single-record entry: Add a single record at a time, filling in the prompts at the command line interface.
2. Batch-mode entry: Supplying the path to a cvs-formatted file to enter multiple records into the database via the command-line interface.

Method 2 is preferred if there are multiple entries to be up-loaded. This method has not gone away. Method 1 is still available as well, but with the introduction of the browser-based UI, using the browser insertion is a quicker and more accurate method for a small handful or records.

Using Musica

Musica is a multi-platform, simple database tool. A traditional SQL database, such as PostgreSQL or MySQL utilizes a single defined database that allows multiple users to access the data. Notes is the graphical interface that allows the user easier access to the data through a web-based UI. Each user is assigned rights/privileges that enable the user to access/modify/add data to the specific database that they are assigned to. If the user is installing Notes/Musica on a domain server, then they will need to have root access. If the personal version of Notes/Musica is being installed, there can be any number of databases and any number of users granted permission to a single database.

If Musica/Notes is being installed on a domain server, it is typically being configured as a curation tool. Any one user can Query the database, but only a db-admin or other approved users can add/modify/edit records. There is a simple security protocol in place to handle this authentication. Authentication is not enabled for personal use. An SQLite database is a single-instance/single-use object database. Each database is accessible by a single user only. Only that user can edit/delete/insert records into their database. This makes SQLite a user-specific tool. The image below demonstrates this philosophy:



Once the database has been configured and tables loaded, you can insert data into Musica. As previously stated, Musica accepts data using one of two methods:

- 1 - using single-entry mode.
- 2 - using Batch-entry mode.

Single-entry mode uses a simple python script located in the Musica/bin path. This script will prompt the user to fill in the appropriate field value. Not all fields will be prompted/populated, however. The fields for Composer, Orchestra or Conductor will only be prompted if the genre of music is Classical or Symphonic. Two examples are shown below:

Simple Record Insertion	Detailed Record Insertion
<pre>% ./musica_db_add_recording_sqlite.py Add New Recording ----- Artist: Metheny Group, Pat Title: The Way Up Year: 2005 Genre (Jazz/Rock/Country/Symphonic): Jazz Format (LP/CD/Cass/RtR/78/4T/8T): CD Label (optional): Nonesuch Catalog Number (optional): 79876-2 Recording Mode (M/S/B) [S]: S Is this a reissue? (y/N): N DBX encoded? (y/N): N Confirm Insert ----- Artist: Metheny Group, Pat Title : The Way Up Year : 2005 Genre : Jazz Format: CD Label : Nonesuch Catalog#: 79876-2 Mode : S Reissue: - DBX : - Proceed? (y/N): y OK: Recording inserted. id: 4</pre>	<pre>./musica_db_add_recording_sqlite.py Add New Recording ----- Artist: Ravel, Maurice Title: Bolero Year: 1984 Genre (Jazz/Rock/Country/Symphonic): Symphonic Composer (optional): Maurice Ravel Orchestra (optional): Orchestrie National De France Conductor (optional): Lorin Maazel Format (LP/CD/Cass/RtR/78/4T/8T): CD Label (optional): CBS Masterworks Catalog Number (optional): MK 37289 Recording Mode (M/S/B) [S]: S Is this a reissue? (y/N): Y Reissue (Y/N): Y DBX encoded? (y/N): N Confirm Insert ----- Artist: Ravel, Maurice Title : Bolero Year : 1984 Genre : Symphonic Composer : Maurice Ravel Orchestra : Orchestrie National De France Conductor : Lorin Maazel Format: CD Label : CBS Masterworks</pre>

	Catalog#: MK 37289 Mode : S Reissue: Y DBX : - Proceed? (y/N): y OK: Recording inserted. Id: 5
--	---

For loading more than a handful (4 or more) record entries or a large csv-formated file listing, batch-entry is the easiest and fastest method of loading data. The `musica_db_batch_load_sqlite.py` utility will read the contents of a named csv-formated file, parse and insert the records, one line at a time, into the Musica database: `% cd ~/Musica/bin % ./musica_db_batch_load_sqlite.py batch_trial_3.csv Musica SQLite Batch`

```
Load ----- DB file : /home/joec/Musica/musica.db
Import dir: /home/joec/Musica/data/imports
CSV file  : /home/joec/Musica/data/imports/batch_trial_3.csv
Table     : recordings
Chunk     : 1000
```

Results

```
-----
Rows before : 0
Rows after  : 190
CSV rows    : 190
Inserted    : 190
Elapsed sec : 0.122746
Rows/sec    : 1547.91
```

The only requirement to use batch loading is that the target batch file have a single header row inserted as the first line of the file:
`artist;title;year;orchestra;conductor;genre;format;label;catalog number;reissue;mode;dbx_encoded`

This will make the typical file look like this:

```
artist;title;year;orchestra;conductor;genre;format;label;catalog number;reissue;mode;dbx_encoded
Roach, Max;Deeds, Not Words;1959;;;Jazz;LP;Riverside Records;RLP 1122;Y;S;
Haydn, Joseph;"Symphony No. 94 "Surprise" · Symphony No. 103 "Drum-Roll"";1960;Philharmonica Hungaria;Dorati,
Antal;Symphonic;LP;Mercury;SR90208;;S;
Krupa, Gene;The Drum Battle (Gene Krupa And Rich, Buddy At JATP);1960;;;Jazz;LP;Verve Records;MG v-8369;Y;S;
```

Batch loading is also useful if you have a discogs catalog that you want to keep local on your network without having to manage the discogs cloud. To retrieve information from your discogs account, you need to perform the following steps:

1. Obtain a discogs API access token -

Navigate to: www.discogs.com and sign in to your account.

Go to **Settings** → **Developers**.

Select **Generate new token** and follow the instructions.

2. Create a JSON configuration file using `template.json` located in `~/Musica/tools`. Copy and save the new json file with your user name.

3. Edit the file using a plain-text editor and populate the fields:

```
{
  "username": "your_discogs_username",
  "token": "your_personal_access_token"
}
```

4. Run `Musica.py` to extract and generate the CSV file.

The resulting output will be placed locally in the directory. The resulting file can be copied/moved to the `~/Musica/data/imports` directory. Check the contents of the extracted file for an misappropriated fields before saving and running the batch loader. NOTE: Due to the nature of the Discogs database, data may not be cleanly stored/retrieved. It is suggested to open the file and validate the contents against what is in discogs and/or your physical media.

Once the data has been loaded, invoke `sqlite3` from a terminal window and issue a simple query to confirm that the data has been loaded and no data errors occurred:

```
sqlite> SELECT artist, year, title, format FROM recordings ORDER BY artist, year, title;
```

artist	year	title	format
Abercrombie, John	1975	Sargasso Sea	LP
Abercrombie, John	1976	Timeless	LP
Ambrosia	1975	Ambrosia	LP
Ambrosia	1976	Somewhere I've Never Travelled	LP
Ambrosia	1978	Life Beyond L.A.	LP

In addition to the script-based methods presented here, data can also be inserted via the sqlite command-line interface (aka CLI).

Note:

Musica is designed to be managed via the provided Python utilities. Direct use of the sqlite3 CLI is supported but not documented in detail here. Users who prefer direct SQL interaction may consult standard SQLite documentation.

Advanced Commands

Once the Musica database has been populated, a user will want to be able to search for a key or field from the database. This requires accessing the database from the sqlite commandline. This is useful if the user wishes to edit/correct a record entry or delete a record entry. To launch the CLI, execute the following commands from the terminal command-line:

```
% cd ~/Musica
% sqlite3 ~/Musica/musica.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .headers on
sqlite> .mode box
sqlite> .nullvalue NULL
sqlite> .mode column
sqlite>
```

This will invoke the sqlite3 application in a terminal window. The next few examples will:

- Check and change the status of the “recording mode” field for a given title -
Title - “The Other Side Abbey Road”
id - 36

From the sqlite command line, enter:

```
SELECT rowid, title, recording_mode
...> FROM recordings
...> WHERE title LIKE '%Other Side%';
36|The Other Side Of Abbey Road|S
```

The output is displayed above, which says that this recording is a Stereo recording (S), but it need to be listed as a Mono (M) recording. So again, using SQL commands, the record id is specified, the recording_mode is changed and the edit is committed.

```

sqlite> BEGIN;
UPDATE recordings
SET recording_mode = 'M'
WHERE rowid = 36;
sqlite> Commit;
sqlite> SELECT rowid, title, recording_mode
FROM recordings
...> WHERE title LIKE '%Other Side%';
36|The Other Side Of Abbey Road|M

```

Querying the database once more shows that the edit has been made. Committed and is now in the database.

- Check the spelling and make a correction for the record label field of a set of record(s)-
There is a suspicion that an entry that is associated with the “Columbia” label may be spelled incorrectly. All Columbia entries need to be verified and corrected if necessary:

```

.headers on
.mode box

```

```

sqlite> SELECT
rowid AS id,
artist,
title,
year
FROM recordings
WHERE label = 'Verve'
ORDER BY artist ASC, year ASC, title ASC;

```

id	artist	title	year	label
21	Blood, Sweat And Tears	Blood, Sweat & Tears Greatest Hits	1972	Columbia
22	Blood, Sweat And Tears	Blood, Sweat & Tears Greatest Hits	1972	Columbia
38	Blood, Sweat And Tears	No Sweat	1973	Columbia
92	Blood, Sweat And Tears	New City	1975	Columbia
126	Blood, Sweat And Tears	More Than Ever	1976	Columbia
5	Chicago	Chicago	1970	Columbia
16	Chicago	Chicago III	1971	Columbia
23	Chicago	Chicago V	1972	Columbia
39	Chicago	Chicago VI	1973	Columbia
62	Chicago	Chicago VII	1974	Columbia

94	Chicago	Chicago VIII	1975	Columbia	
128	Chicago	Chicago X	1976	Columbia	
153	Chicago	Chicago XI	1977	Columbia	
186	Chicago	Hot Streets	1978	Columbia	
159	Davis, Miles/Dameron, Tadd Quintet,	In Paris Festival International De Jazz - May, 1949	1977	Columbia	
161	Diamond, Neil	Love At The Greek: Recorded Live At The Greek Theatre	1977	Columbia	
162	Eath, Wind & Fire	All 'N All	1977	Columbia	
68	Ferguson, Maynard	Chameleon	1974	Columbia	
69	Ferguson, Maynard	M.F. Horn 4&5: Live At Jimmy's	1974	Columbia	
163	Ferguson, Maynard	New Vintage	1977	Columbia	
164	Ferguson, Maynard	Conquistador	1977	Columbia	
27	Fogelberg, Dan	Home Free	1972	Columbia	
166	Gale, Eric	Ginseng Woman	1977	Columbia	
99	Getz, Stan	Captain Marvel	1975	Columbia	
71	Hancock, Herbie	Thrust	1974	Columbia	
139	Joel, Billy	Turnstiles	1976	Columbia	
75	Lewis, Ramsey	Sun Goddess	1974	Columbia	
110	Simon, Paul	Still Crazy After All These Years	1975	Columbia	
111	Springsteen, Bruce	Born To Run	1975	Columbia	
181	Various	Montreux Summit, Volume 1	1977	Columbia	
148	Weather Report	Black Market	1976	Columbia	
182	Weather Report	Heavy Weather	1977	Columbia	
106	Rampal, Jean-Pierre	Suite For Flute And Jazz Piano	1975	Columbia Masterworks	
100	Haydn, Joseph	Quartets, Opus 76 (Complete)	1975	Columbia Odyssey	
179	Tchaikovsky, Pyotr Ilyich	Symphony No. 6 In B Minor, Op. 74 ("Pathetique")	1977	Columbia Special Products	

- Perform a query on the entire database and sort by ALL the First Letter “A”

.headers on

.mode box

SELECT

artist,

title,

year,

format,

label

FROM recordings

WHERE artist LIKE 'A%'

AND format = 'LP'

ORDER BY artist ASC, year ASC, title ASC;

artist	title	year	format	label
Abercrombie, John	Sargasso Sea	1975	LP	ECM Records
Abercrombie, John	Timeless	1976	LP	ECM Records
Ambrosia	Ambrosia	1975	LP	20th Century Records
Ambrosia	Somewhere I've Never Travelled	1976	LP	20th Century Records
Ambrosia	Life Beyond L.A.	1978	LP	Warner Bros. Records
America	Holiday	1974	LP	Warner Bros. Records

- Exporting Data is similar to performing a query, except the data is written to a CSV file. EXPORT is a read-only operation. It simply reads the database, confirms the requested data conforms to the SQL request and saves the data to a CSV file. To export a file, the database needs have to have formatting constraints enabled.

```
.headers on
.mode csv
.separator ";"
```

The preceding commands are used to format database output. In this instance:

Column headers are enabled.

The output mode is set to CSV formatted file.

The default field delimiter is set to a semicolon “;”.

The .output formatting command specifies the target file.

```
.output /home/<user>/Musica/data/exports/crusaders_export.csv
```

Note – for the output command, you MUST use the full path length. Sqlite DOES NOT expand the ‘~’ for the user.

The actual query is no different than any other query:

```
SELECT
artist,
title,
year,
label,
format,
recording_mode,
reissue
FROM recordings
WHERE artist LIKE '%Crusaders%' COLLATE NOCASE
ORDER BY artist, year, title;
```

The command is then completed by resetting the output:

```
.output stdout
```

After the command is completed, the program will return a system prompt. The exported data is shown below:

```
% cat crusaders_export.csv
artist;title;year;label;format;recording_mode;reissue
"Crusaders, The";"Old Socks, New Shoes...New Socks, Old Shoes";1970;Chisa;LP;S;
"Crusaders, The";"The Best Of The Jazz Crusaders";1970;"World Pacific Jazz";LP;S;
"Crusaders, The";"Crusaders 1";1972;"Blue Thumb Records";LP;S;
"Crusaders, The";Hollywood;1972;MoWest;LP;S;
"Crusaders, The";"The 2nd Crusade";1973;"Blue Thumb Records";LP;S;
"Crusaders, The";"Unsung Heroes";1973;"Blue Thumb Records";LP;S;
"Crusaders, The";Scratch;1974;"Blue Thumb Records";LP;S;
"Crusaders, The";"Southern Comfort";1974;"Blue Thumb Records";LP;S;
"Crusaders, The";"The Young Rabbits";1975;"Blue Note";LP;S;
"Crusaders, The";"Those Southern Knights";1976;"ABC Blue Thumb";LP;S;
"Crusaders, The";"Free As The Wind";1977;"ABC Blue Thumb";LP;S;
"Crusaders, The";"Chain Reaction";1978;"Mobile Fidelity Sound Lab";LP;S;Y
"Crusaders, The";Images;1978;"ABC Blue Thumb";LP;S;
```

Notes

What is Notes?

Notes is the browser-based graphical interface that interacts with the sqlite3 musica.db database. It is a tool that allows users to Query, Add, Edit, Delete a record from the database without having to learn cryptic SQL commands in order to access/modify their music collection. Notes does not replace SQL, it simply allows another way to interact with the database. It is still easier, however, to batch-load a large collection of records via the batch-add utility than it is via Notes. Notes is used to Query/Modify a database one record at a time.

Using Notes

Note: Directory paths shown in examples use Linux/macOS notation. Windows users should substitute the appropriate Windows path separator (\).

There are two ways to use Notes:

1. **Domain-based Deployment:** As a domain-based browser front-end, where a web-server such as Apache is available and is serving pages on an internal or public network (commercial record store).

To use Notes from a centralized domain server, refer to the installation guide “**Notes Install V1.0**” as part of the application download. To launch Notes as a dedicated domain service, open a browser and enter the following command in the browser URL-field¹:

www.musica-notes.com

2. **Local/private network deployment:** As a localized application on a private network. To use Notes from a personal desktop device (desktop workstation or laptop), refer to the “**Notes Install V1.0**” as part of the application download. For local use, the Python web server must be running so that Notes can communicate with the musica.db database..

To launch Notes on a personal computing platform, execute the following commands:

- from a terminal window (terminal/xterm in linux/MacOS or terminal in Windows), change directories to the location of the musica.db that was installed. Typically `cd /home/<user_id>/Musica`
- Navigate to the directory web/py - `cd ./web/py`

¹ - The URL specified here is just a suggestion.

- Locate the executable file named server.py. Execute the file from the command line. This will launch the python server which will enable the Notes page to interact with the SQL back-end:
python ./server.py

After the server is launched, the following information is displayed:

Musica-Notes standalone server

Listening on:

<http://127.0.0.1:8000/>

Press Ctrl-C to stop.

Open a browser and enter the above http address in the URL-field.

The Notes front page for notes will appear² -

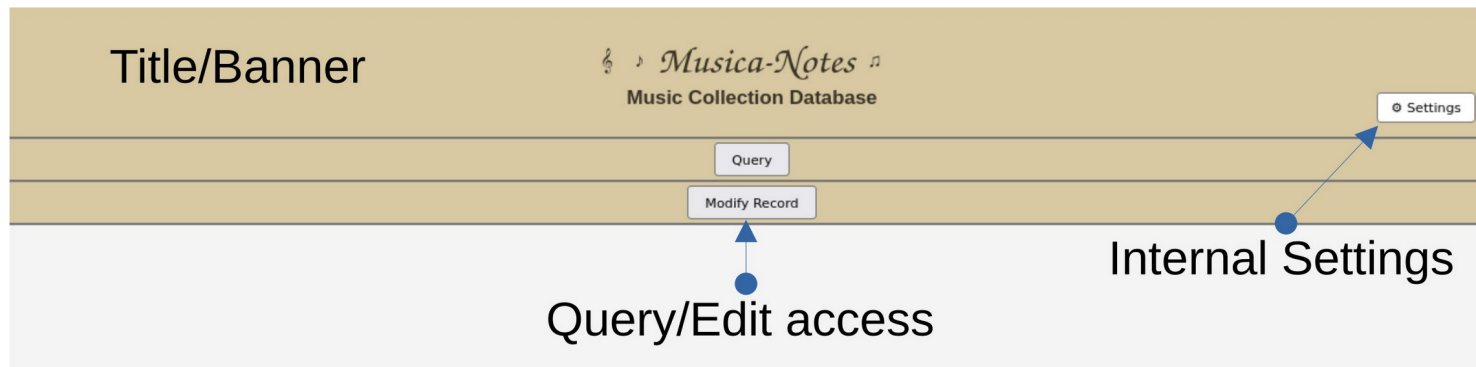


Fig 1 – Notes Front page

There are three buttons associated on the front page:

2 - If the page is blocked by the domain server or a traceback error is displayed, refer to the Troubleshooting section at the end of this guide.

- **Settings:** Displays the configuration values established during installation and stored in the `config.dta` file (see the *Notes Installation Guide*). Two Settings dialogs are available, depending on the selected usage model. The model below shows the ‘Commercial’ information (Fig 2a). The Commercial usage model is shown in Fig. 2a. The Personal usage model is shown in Fig. 2b.:

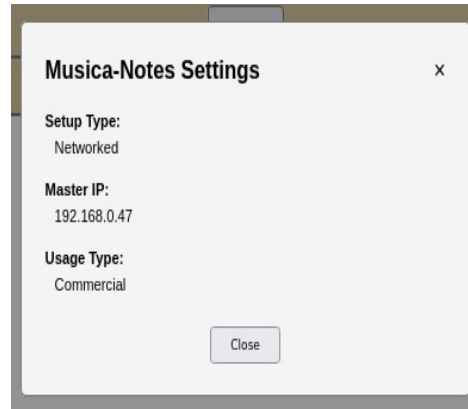


Fig 2a – Settings Dialog (Commercial)

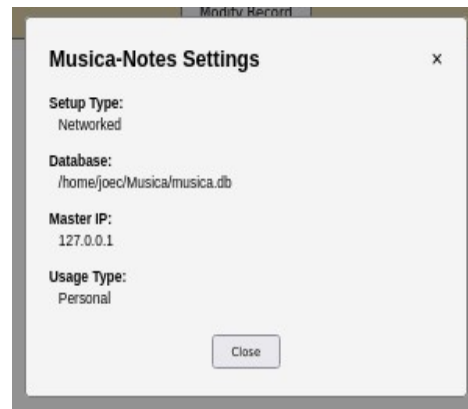
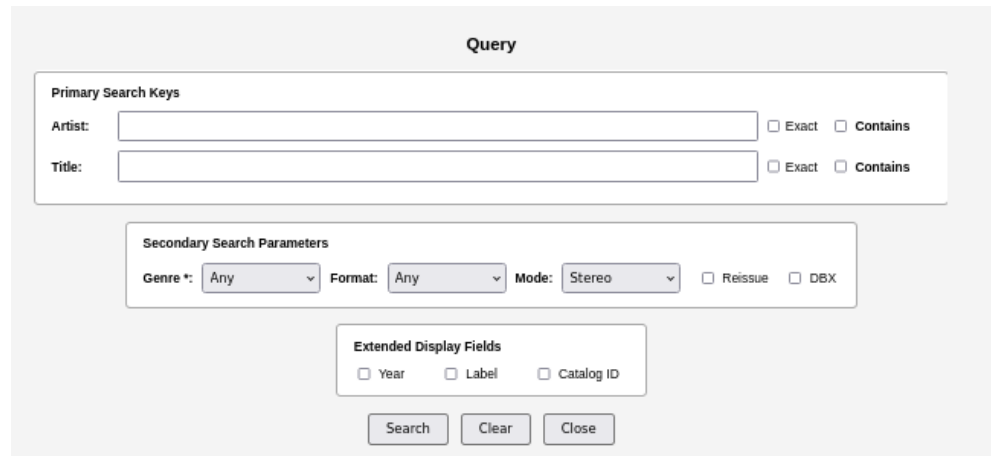


Fig 2b – Settings Dialog (Personal)

The difference between the two is that the database path is displayed on the Personal version. For a Commercial usage model, casual users have no need to know the location of the database. For a Personal user, it is helpful to know where the database resides. Omitting the

database path from the Commercial version also prevents unnecessary disclosure of server filesystem information.

- Query: access and query the contents of the database



The screenshot shows a web-based 'Query' form. It is divided into three main sections: 'Primary Search Keys', 'Secondary Search Parameters', and 'Extended Display Fields'. The 'Primary Search Keys' section contains two text input fields, 'Artist:' and 'Title:', each followed by 'Exact' and 'Contains' checkboxes. The 'Secondary Search Parameters' section includes three dropdown menus for 'Genre *', 'Format', and 'Mode', and two checkboxes for 'Reissue' and 'DBX'. The 'Extended Display Fields' section has three checkboxes for 'Year', 'Label', and 'Catalog ID'. At the bottom are three buttons: 'Search', 'Clear', and 'Close'.

Fig 3a – Query Form (basic)

The Query form consists of the following fields and is subdivided into three blocks:

Primary Search Keys

Primary Search Keys are the primary fields used to identify records when a query is performed. At a minimum, the Rec ID, Artist Name, Recording Title, and Genre will always be returned in the query results.

- Artist – By default, the form uses a “fuzzy” wild-card search. If the user knows the exact artist name, select the “Exact” check-box. If they know part of the Artist’s name, then they can check the ‘Contains’ box.
- Title – By default, the form also uses a “fuzzy” wild-card search. If the user knows the exact title name, select the “Exact” check-box. If they know part of the title, then they can check the ‘Contains’ box.

Note: The Exact/Contain/’fuzzy’ search logic is used across many dialogs (Query, Edit, Delete). This is an Exclusive-Or search mechanism and will be demonstrated fully later in this document.

Secondary Search Parameters

With the exception of Genre (which is always returned), the other parameters are ‘Secondary’ or ‘Enhanced Information’ fields. These allow the user to retrieve more information around any record that is queried and displayed.

- Genre – A pulldown that restricts the query to one of the specified genres. The default is “Any” and will retrieve any genre in the database. The basic Query dialog that is shown is the ‘generic’ used for Jazz, Rock and Country. If the user selects Classical, Symphonic or Soundtrack, then the enhance Query form is displayed



The image shows a screenshot of a 'Query' dialog box. At the top, the title 'Query' is centered. Below it, there is a section titled 'Primary Search Keys'. This section contains five rows of search fields, each with a label on the left, a text input box in the middle, and two checkboxes on the right. The labels are 'Artist:', 'Title:', 'Composer:', 'Conductor:', and 'Orchestra:'. The checkboxes are labeled 'Exact' and 'Contains'. All input boxes are empty, and all checkboxes are unchecked.

Primary Search Keys	Exact	Contains
Artist:	☐	☐
Title:	☐	☐
Composer:	☐	☐
Conductor:	☐	☐
Orchestra:	☐	☐

These three genres often have a Composer, Conductor and Orchestra associated with the record.

- Format – A pulldown that restricts the query to one of the specified formats – Any, LP, CD, Cassette, RTR (Reel-to-Reel), 78RPM, 4-track, 8-track. The default is “Any” and will retrieve any format from the database.
- Mode – This pulldown restricts the query to Stereo or Mono recordings. The default mode is “**Stereo**”, although some compilation/reissue recordings have a mix of Mono and Stereo, so “**Both**” is listed as an option.

- Reissue – Select this checkbox to restrict the query to records identified as reissues.
- DBX – Select this checkbox to restrict the query to records identified as DBX-encoded recordings or tapes.

Extended Display Fields

Extended Display Fields are simply extra information that will be displayed in the query results:

- Year – this is the year the record was ‘released’.
- Label – The label associated with the listed recording. It is also useful to distinguish between original pressings and reissues.
- Catalog ID – Another useful field that can assist in distinguishing between an original and a reissue.

Modify Record

If Notes used as a part of a commercial enterprise or of the administrator prefers to maintain control of the database, then the user will need to sign-in and be on the approved user list to gain access. Therefore, when Modify Record is pressed, the Login or Data Access screen will appear:

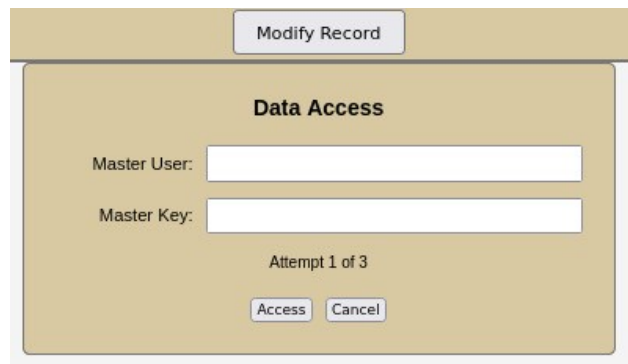


Fig 4 – Login Screen

The Login Screen requires a Master User and a Master Key (aka Passwd) to access/edit records in the database. Instructions on how to setup/configure can be found in the the Notes Installation Guide. Once the user signs in (or if Notes is configured for Personal use), then

the Modify Record controls become visible. If Notes is configured as part of a Personal access tool, the Security is not present and will not be shown³.

Modify Records

The Modify Record banner consists of three buttons: Add Record, Edit Record, and Delete Record. Each button opens its associated dialog. The Modify Record screen remains active until the user selects the Query button.

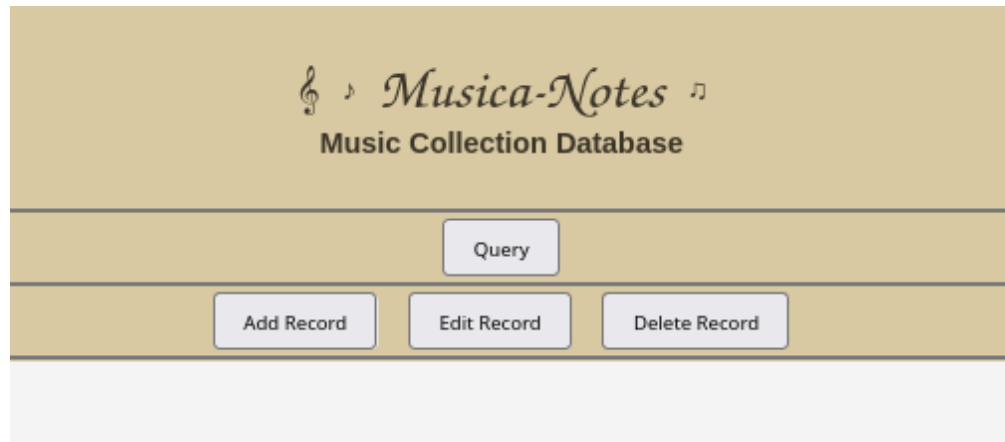
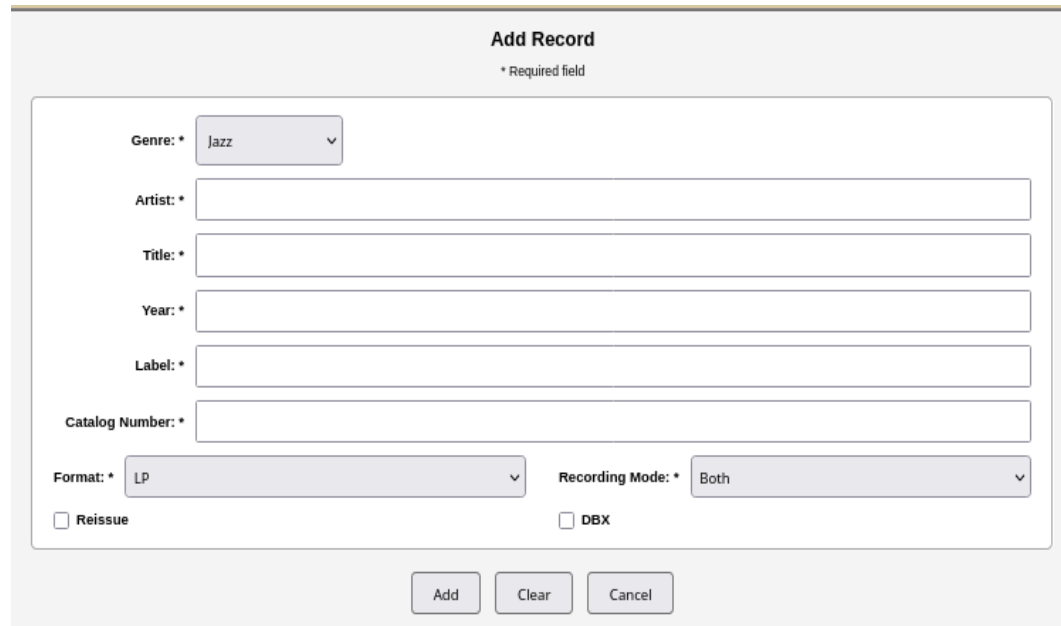


Fig 5 - Modify Records Screen

³ If I user wishes to change the usage model from Personal to Private and exert access control, refer to the Notes-Install_V1.0 document on setting up a Private database and vice-versa.

Add Record



The image shows a software dialog box titled "Add Record". At the top, below the title, is a small note: "* Required field". The dialog contains several input fields arranged vertically. The first field is a dropdown menu labeled "Genre: *" with "Jazz" selected and a downward arrow. The next four fields are text boxes labeled "Artist: *", "Title: *", "Year: *", and "Label: *". The fifth field is a text box labeled "Catalog Number: *". Below these are two more dropdown menus: "Format: *" with "LP" selected, and "Recording Mode: *" with "Both" selected. At the bottom of the input area are two checkboxes: "Reissue" and "DBX". At the very bottom of the dialog are three buttons: "Add", "Clear", and "Cancel".

Add Record
* Required field

Genre: * Jazz ▾

Artist: *

Title: *

Year: *

Label: *

Catalog Number: *

Format: * LP ▾

Recording Mode: * Both ▾

☐ Reissue ☐ DBX

Add Clear Cancel

Fig 6a – Basic Add Record dialog

The Add Record dialog contains the fields required to add a record to the musica database.. The key field is the Genre pulldown. There are 6 genres available for selection:

- Jazz
- Rock
- Country
- Classical
- Symphonic
- Soundtrack

The dialog shown above is the Basic Add Record dialog. It appears for all genres except Classical, Symphonic, and Soundtrack. When one of these genres is selected, the Enhanced Add Record dialog appears.



The image shows a software dialog box titled "Add Record". At the top, it says "* Required field". The dialog contains several input fields and checkboxes. The "Genre" field is a dropdown menu currently set to "Classical". Below it are text input fields for "Artist:", "Title:", "Year:", "Composer:", "Conductor:", "Orchestra:", and "Label:". Below these is a "Catalog Number:" field. At the bottom left, there is a "Format:" dropdown menu set to "LP" and a checkbox labeled "Reissue". To the right of the "Format" dropdown is a "Recording Mode:" dropdown menu set to "Both" and a checkbox labeled "DBX". At the very bottom of the dialog are three buttons: "Add", "Clear", and "Cancel".

Fig 6b – Enhanced Add Record dialog

As shown, the Enhanced dialog contains fields for Composer, Conductor and Orchestra as befitting a Classical/Symphonic/Soundtrack recording. This allows the user to differentiate between different versions of *Night on Bald Mountain* by Conductor and Orchestra, while the Composer (Mussorgsky) remains common to the recordings. Fields marked with an asterisk are required fields. The Composer, Conductor, and Orchestra fields are displayed for Classical, Symphonic, and Soundtrack recordings. For other genres, these fields are hidden and are not populated in the database.

The Format pulldown provides the available recording formats (LP is the default format):

- LP
- CD
- Cassette
- RtR (Reel-to-Reel tape)
- 78 RPM (Shellac recordings, pre-LP recordings)
- 4-track
- 8-track

One of these formats need to be selected to fill out the dialog.

The two remaining fields, Reissue and DBX, are optional. Both provide additional information about the recording. This allows users to list recordings in their collection that are DBX-encoded. It is for those collectors who have a DBX encoder/decoder to play back the entry in their collection or, if a tape has been recorded using DBX and thus needs to be played back via a DBX decoder. Reissue is useful to track which recordings are reissues rather than original pressings or tapings.

Modify Record

The Edit Record dialog is similar in look and feel to the Query dialog. The Edit Record dialog provides Artist and Title search fields, along with a Genre selection. the user can click the “Exact” box or the Contains field if the Artist/Title is a portion of the record.

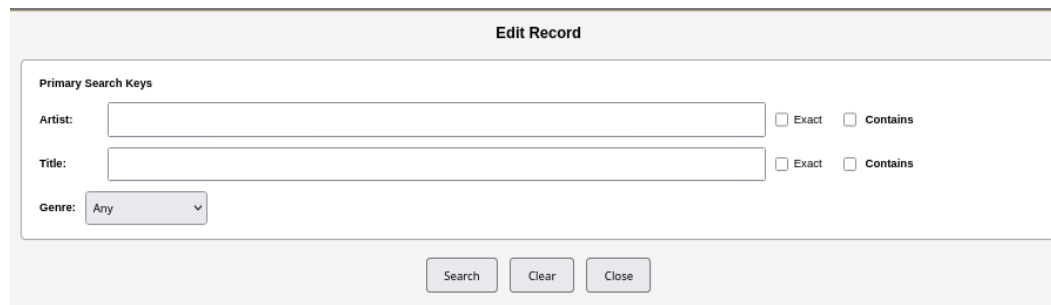


Fig 8 – Edit Record (Query) dialog

In the example above, the artist to search for is “Barron”. Since we want ALL artists with “Barron” in the name, the ‘Contains’ checkbox will be selected before pressing the ‘Search’. The resulting records are shown below:

Query

Primary Search Keys

Artist: ☐ Exact ☒ Contains

Title: ☐ Exact ☐ Contains

Secondary Search Parameters

Genre *: Format: Mode: ☐ Reissue ☐ DBX

Extended Display Fields

☐ Year ☐ Label ☐ Catalog ID

Results

Rec ID	Artist	Title	Genre
966	Barron, Kenny	Live at Maybeck Recital Hall, Volume Ten	Jazz
967	Barron, Kenny	Other Places	Jazz
964	Mitchell, Red/Barron, Kenny	The Red Barron Duo	Jazz

Fig 9 – Edit Results form

Three records are returned. A horizontal scrollbar at the bottom of the results allows the user to view information that extends beyond the visible results area. Once the desired record has been identified, click anywhere on the record row. At that point, the Edit dialog will appear (see below).

Edit Record

Record Information

Rec ID:

967

Genre:

jazz

▼

Year:

1993

Artist:

Barron, Kenny

Title:

Other Places

Composer:

Conductor:

Orchestra:

Label:

Verve

Catalog Number:

314 519 699-2

Format:

CD

▼

Recording Mode:

Stereo

▼

☐ Reissue

☐ DBX

Save Changes

Cancel

Fig 10 – Edit Record (result) dialog

The user can then make the desired changes to the record and select either “Save Changes” or “Cancel”. If the user selects “Save Changes”, a confirmation dialog asks the user to confirm the changes. If the user selects “Cancel”, the user is returned to the previous dialog (this action is identical across all dialog forms). If the user selects “OK”, the dialog closes and the results are updated immediately.

Delete Record

Delete Record allows the user to delete a record from the current database. To select a record for deletion, select the Delete Record button. This displays the Delete Record dialog.



The image shows a 'Delete Record' dialog box. It has a title bar at the top that says 'Delete Record'. Below the title bar is a section titled 'Primary Search Keys'. This section contains three input fields: 'Artist:', 'Title:', and 'Genre:'. The 'Artist:' and 'Title:' fields are text boxes, and the 'Genre:' field is a dropdown menu currently showing 'Any'. To the right of the 'Artist:' and 'Title:' text boxes are two checkboxes each, labeled 'Exact' and 'Contains'. Below the search fields are three buttons: 'Search', 'Clear', and 'Close'.

Fig 11 – Delete Record dialog

The Delete dialog

The Delete Record dialog works almost identically to the Edit Record dialog, with one exception: the user must specify an Artist or Title before a query can be performed. This keeps the search focused and reduces the possibility of accidentally deleting an unwanted record by requiring the user to identify a specific record before deletion. The user can select “Delete” or “Cancel”. If “Delete” is selected, the selected record is immediately removed from the database. If “Cancel” is selected, no action is taken and the user can select another record or choose another operation.

Delete Record

Primary Search Keys

Artist: Will

☐ Exact ☐ Contains

Title:

☐ Exact ☐ Contains

Genre: Soundtrack

▼

Search

Clear

Close

Results — select a record to delete

Rec ID	Artist	Title	Year	Composer	Orchestra	Conductor	Genre	Format	Label	Catalog
869	Williams, John	Close Encounters Of The Third Kind (Original Motion Picture Soundtrack)	1977				Soundtrack	LP	Arista	AL 950
868	Williams, John	Star Wars	1977				Soundtrack	LP	20th Century Records	2T-541

Confirm Delete

Delete Record 869 — Williams, John / Close Encounters Of The Third Kind (Original Motion Picture Soundtrack)?

Delete

Cancel

Fig 12 – Delete Confirmation dialog

The user can select Delete or Cancel. If Delete is chosen, then the selected item is immediately removed from the database. If the user selects Cancel, then nothing happens and the user is free to choose another record or chose another operation.

Specific Notes Examples

Below are a few examples to give the user an idea of how to use some of the more complicated dialog boxes in Notes.

Query

The simplest example to start off with is the Query command. The Query dialog prompts the user to supply search information. Notes uses this information to construct the SQL query and retrieve the matching records from the database. The results are displayed below the dialog.

- Example 1: General Query – Artist

Query

Primary Search Keys

Artist: ☐ Exact ☐ Contains

Title: ☐ Exact ☐ Contains

Secondary Search Parameters

Genre *: Format: Mode: ☐ Reissue ☐ DBX

Extended Display Fields

☐ Year ☐ Label ☐ Catalog ID

Results

Rec ID	Artist	Title	Genre
111	Burton, Gary	Turn Of The Century	Jazz
112	Burton, Gary Quartet	Picture This	Jazz
113	Burton, Gary Quintet	Ring	Jazz

Fig 13a – Search for Burton

In this example, the user enters “Burton” in the Artist field and selects Jazz from the Genre pulldown. The Exact and Contains options are not selected, so the default search method is used. After selecting Search, Notes returns the records matching the search criteria. Three records are returned: Gary Burton, Gary Burton Quartet, and Gary Burton Quintet.

- Example 2 – Specify Artist by name

In this example, the Artist name being searched for is Burton. The search specifies that “Burton” must appear at the beginning of the Artist name. When “Search” is pressed, three records are returned: one for a solo recording and the other two for recordings by a Quartet. The SQL command string is presented here:

```
SELECT id, artist, title, genre FROM recordings WHERE artist GLOB 'Burton*' AND genre = 'Jazz' ORDER BY artist ASC, year ASC;
```

The Query uses the SQLite GLOB operator for this type of search. In this instance, **Burton*** specifies that “Burton” must appear at the beginning of the Artist name. The * is a wildcard that allows any characters to follow “Burton”. Changing the search criteria to specify the artist’s full name and selecting “Exact” changes the returned results:

Query

Primary Search Keys

Artist: ☒ Exact ☐ Contains

Title: ☐ Exact ☐ Contains

Secondary Search Parameters

Genre *: Format: Mode: ☐ Reissue ☐ DBX

Extended Display Fields

☐ Year ☐ Label ☐ Catalog ID

Results

Rec ID	Artist	Title	Genre
111	Burton, Gary	Turn Of The Century	Jazz

Fig 13b – Search narrowed to specific Artist name

The SQL command is now written to use the ‘Exact’ checkbox, which changes the SQL syntax:

```
SELECT id, artist, title, genre FROM recordings WHERE artist = 'Burton, Gary' AND genre = 'Jazz' ORDER BY artist ASC, year ASC;
```

The Artist search now returns only records with an exact match for the specified artist name.

- Example 3 – Match Artist in a String

If the user wants a broader match against all records that contain a given artist name, the “Contains” checkbox should be selected.

The screenshot shows a web-based search interface titled "Query". It contains three main sections: "Primary Search Keys", "Secondary Search Parameters", and "Extended Display Fields".

Primary Search Keys: This section has two input fields. The "Artist:" field contains the text "Burton, Gary". To its right are two checkboxes: "Exact" (unchecked) and "Contains" (checked). The "Title:" field is empty. To its right are two checkboxes: "Exact" (unchecked) and "Contains" (unchecked).

Secondary Search Parameters: This section contains three dropdown menus: "Genre *" set to "Jazz", "Format:" set to "Any", and "Mode:" set to "Stereo". To the right of these are two checkboxes: "Reissue" (unchecked) and "DBX" (unchecked).

Extended Display Fields: This section contains three checkboxes: "Year" (unchecked), "Label" (unchecked), and "Catalog ID" (unchecked).

Below these sections are three buttons: "Search", "Clear", and "Close".

Results: Below the buttons is a table with the following data:

Rec ID	Artist	Title	Genre
111	Burton, Gary	Turn Of The Century	Jazz
112	Burton, Gary Quartet	Picture This	Jazz
113	Burton, Gary Quintet	Ring	Jazz

Fig 13c – Search broadened for any occurrence of Artist name

From this example, the SQL query is written to search for the artist name simply being included in the Artist field:

```
SELECT id, artist, title, genre FROM recordings WHERE artist GLOB '*Burton*' AND genre = 'Jazz' ORDER BY artist ASC, year ASC;
```

The SQL query uses the SQLite GLOB operator with wildcards to search the entire Artist field for the specified search string. The * wildcard allows any characters to appear before or after the search string.

Add Record

Adding a record inserts a new item into the database. There are, however, rules that apply when inserting a record.

- Fields marked with a ‘*’ are required. If a field is blank and the user presses ‘Add’, then a popup will appear informing the user that the field cannot be left blank. The only exception is when adding a Soundtrack recording, as explained below.
- Reissue and DBX are both optional fields. They provide additional information to identify a re-released recording or a recording that has been DBX encoded and, as a result, requires a DBX decoding device.
- Example 1 – Add Record to the Database

In this example, a new record will be inserted into the database. Click the Add Record button to access the Add Record dialog box. For this example, Jazz, Symphonic and Soundtrack recordings will be inserted. The first ‘record’ to be added will be the Jazz entry. The Artist Name, Title, Year of Release, Label, Catalog Number, Format and Recording Mode are typically filled in from the recording information⁴. The Reissue will come from either the jacket information (by listing multiple years of releases) or from information found from online sites (a la Discogs). Once the information is filled in to the user’s liking, press ‘Add’. Fields marked with an ‘*’ are required fields. If a field is left blank, a message will appear warning the user when they attempt to save/insert the record into the database. If the user doesn’t have the necessary information, they can work around the text field by typing “NA” or “-” or use the default value for the pulldown. At a later time, they can Edit the record with the correct information. Once all of the information fields are filled in and the user clicks “Add”, a popup will appear informing the user of the addition along with the Record ID⁵. The Add Record dialog will clear itself for the next entry.

4 - Artist information is typically added in a Last Name, First Name format or using the Artist colloquial name (Davis, Miles/The Rippingtons/Professor Longhair, etc)

5 - Records are inserted on a First come/First In basis, regardless of Artist’s Name or year of release. Musica handles the data display internally.

Add Record

* Required field

Genre: *

jazz

▼

Artist: *

Professor Longhair

Title: *

Longhair's Blues

Year: *

2000

Label: *

Bluenote

Catalog Number: *

BN 5409-CD

Format: *

CD

▼

Recording Mode: *

Mono

▼

☒ Reissue

☐ DBX

Add

Clear

Cancel

Fig 14a – Add Record (Jazz)

- Example 2 – Add Symphonic Record

For the Symphonic entry, the Genre pull-down is set to ‘Symphonic’. Once Symphonic is selected, the Add Record dialog will expand to include Composer, Orchestra and Conductor. As above for the Jazz entry, these are all required fields and, if a user attempts to add a recording without all of the marked fields filled, a warning popup will appear and force the user to fill in any blank fields that exist.

Add Record
* Required field

Genre: * Symphonic ▾

Artist: * Copland, Aaron

Title: * Grand Canyon Suite/Appalachian Spring

Composer: * Copland, Aaron

Conductor: * Copland, Aaron

Orchestra: * Dallas Philharmonic

Year: * 2005 Label: * Seraphim Catalog Number: * SLP-80089

Format: * LP ▾ Recording Mode: * Stereo ▾ ☐ Reissue ☒ DBX

Add
Clear
Cancel

Fig 14b – Add Record (Symphonic)

- Example 3 – Add Soundtrack Record

The 3rd and final example is creating a Soundtrack record entry. The Soundtrack entry is similar to both the Classical and Symphonic recordings, except the fields for Composer/Conductor/Orchestra are optional. For example:

Field	Soundtrack 1	Soundtrack 2
Artist	Williams, John	Various
Title	Close Encounters Of The 3 rd Kind	Flashdance
Composer	Williams, John	NA or blank
Conductor	Williams, John	NA or blank
Orchestra	Boston Pops Symphony Orchestra	NA or blank

From the table, there are two sample Soundtrack records, each one has different information. Soundtrack 1 is a symphonic-style entry with a Composer/Conductor/Orchestra. Soundtrack 2 is closer to a 'Greatest Hits' compilation. As a result, when a Soundtrack record is added, the Composer/Conductor/Orchestra fields may appear, but they can be left blank and a warning popup will not appear when Add is pressed.

The screenshot shows a web form titled "Add Record" with a subtitle "* Required field". The form contains several input fields and checkboxes. The "Genre" field is a dropdown menu set to "Soundtrack". The "Artist" field contains "Various". The "Title" field contains "Flashdance". The "Composer" field contains "NA". The "Conductor" and "Orchestra" fields are empty. The "Year" field contains "1985", the "Label" field contains "Arista", and the "Catalog Number" field contains "AR-5609". The "Format" field is a dropdown menu set to "LP". The "Recording Mode" field is a dropdown menu set to "Stereo". There are two checkboxes, "Reissue" and "DBX", both of which are unchecked. At the bottom of the form are three buttons: "Add", "Clear", and "Cancel".

Genre: *	Soundtrack	
Artist: *	Various	
Title: *	Flashdance	
Composer: *	NA	
Conductor: *		
Orchestra: *		
Year: *	Label: *	Catalog Number: *
1985	Arista	AR-5609
Format: *	Recording Mode: *	☐ Reissue ☐ DBX
LP	Stereo	

Add Clear Cancel

Fig 14c – Add Record (Soundtrack)

Troubleshooting - Musica

If during the course of using Musica, one or all of the following issues arise, it is a matter of simple troubleshooting to detect/find/correct the problem.

Symptom	Common Cause(s)	Resolution
No rows returned from a query	Misspelled value Case mismatch Extra whitespace Table is empty	<code>SELECT COUNT(*) FROM recordings;</code>
		<code>SELECT DISTINCT label FROM recordings;</code>
		<code>SELECT DISTINCT label FROM recordings;</code>
Data appears “missing”	Check spelling variations	<code>SELECT DISTINCT artist FROM recordings WHERE artist LIKE '%Davis%' COLLATE NOCASE;</code>
UPDATE changed more rows than expected	Always run the SELECT first:	<code>SELECT rowid, artist, title FROM recordings WHERE label = 'Incorrect Label';</code>
	The update inside a transaction:	<code>BEGIN; UPDATE recordings SET label = 'Correct Label' WHERE label = 'Incorrect Label'; COMMIT;</code>
	If unsure:	<code>ROLLBACK;</code>
Parse error: no such table: recordings	Confirm sqlite wasnt launched from the proper location.	<code>sqlite> .quit % cd ~/Musica % sqlite3 musica.db sqlite></code>
INSERT fails	Column mismatch Wrong number of values Typo in column name	<code>PRAGMA table_info(recordings);</code>
		<code>.schema recordings</code>
Nothing is working	Reset the session	<code>Sqlite3> ~/Musica/musica.db .headers on .mode box SELECT COUNT(*) FROM recordings;</code>

CLI Formatting looks strange	.mode box renders poorly (older terminal):	.mode column .headers on
------------------------------	--	-----------------------------

Troubleshooting – Notes

In launching/using Notes, one or more of the following problems may arise. These are the steps to resolve them:

Symptom	Common Cause(s)	Resolution
Attempting to launch www.musica-notes.com results in a blocked message if running an ad-blocker/DNS sinkhole	The domain is seen by the ad-blocker as an untrustworthy site.	From the ad-blocker, mark the domain www.musica-notes.com as a safe site.
When launching Notes from a browser, the user sees “Unable to connect”	The domain server is offline	Confirm the server is up and available to serve content
	The network is down	If the domain server is online, check the the network cables are plugged in
Receive an error message popup -”Unable to execute query”	The database file is either missing or has yet to be created.	Check the ‘Settings’ button to determine where the database file should be installed. Then from the command line or file browser (if using Windows), check to see if musica.db exists.
Looks like there’s a problem with your internet connection.	The server is not available.	Check that the internet cables are plugged
Can’t connect to	The server is not available or cannot be found.	Try again later
		Check your network connection

<specified_server>		Check that the browser has permission to access the web (you might be connected but behind a firewall)
Attempting to launch the python web-server from the command line generates the following error: OSError: [Errno 98] Address already in use	There is already a version/process of server.py running.	Either kill the existing process, or just launch the Notes UI in a browser using http://127.0.0.1:8000
Unable to open the file server.py	can't open file '../server.py': [Errno 2] No such file or directory	Confirm that the correct executable path for server.py is being used. See the installation instructions for Personal Notes.

Appendix A: SQL Database User Examples

As stated previously, SQLite is a widely deployed, public-domain software library that provides a self-contained, server-less, zero-configuration, transactional SQL database engine. Unlike traditional client-server database systems such as MySQL or PostgreSQL, SQLite runs within the host application's process and stores the entire database in a single, cross-platform disk file. It does behave identically to other more traditional SQL back-end tools. As a C-based back-end, users have control over how the sqlite UI is presented. A full set of examples can be found [online](#), but for now, a sampling of these commands include:

SQLite UI/UX command table			
Command	Definitions	Command	Definitions
<code>.headers on/off</code>	Enables headers for data output.	<code>.bail <on off></code>	Controls how sqlite behaves when it encounters an error.
<code>.mode <box column etc></code>	Instructs sqlite on how to display data.	<code>.echo on off</code>	Echos commands typed when using the interactive CLI before the command is executed.
<code>.nullvalue "..."</code>	Tells sqlite how to interpret/display a value for NULL value.	<code>.show</code>	Display values for various settings.
<code>.width W1 W1... WN</code>	Instructs sqlite on how to format the columns to display data.	<code>.save ?OPTIONS? FILE</code>	Write the current database to a file.
<code>.sqliterc</code>	User-created control file. Located in the users home directory (i.e. <code>~/<user>/.sqliterc</code> for *nix/MacOS or <code>C:\Users\<username>\.sqliterc</code> for Windows).	<code>.timer on off</code>	Turn the SQL timer on or off

By taking advantage of a control file, many setup commands can be loaded/executed when sqlite3 is launched. The control file works across all OS platforms. This means preferred settings on one platform can be carried over to another with no loss of look and feel. The information

presented above is useful when using SQLite from the CLI. The control-commands are only persistent for a single session, unless a `.sqliterc` file is created/read-in with each invocation of the `sqlite` program.

Appendix B: Detailed Commands and Usage

The examples below expand on the previous commands in Appendix A as well as the Main document. These are meant to be used as references, whereas the Main document is meant to enable the user/admin to become familiar enough with the tool to begin using it after a few minutes of setup. Below are solid, explicitly written command scenarios to setup/modify and use Musica for both the Admin and/or users. Refer to the table in **Appendix A: SQL Database User Examples**

Clearing the terminal window

At some point, the terminal window/screen has output from several executed commands, or errors from poorly formed commands (don't worry, it happens). To refresh the screen to just a DB prompt, enter the following command sequence:

- Move the mouse pointer into the window executing the Maria/MySQL program and 'left-click' the mouse.
- From the keyboard, type/enter "<Ctrl – L>"

This will clear the screen and return the database prompt. It will also clear the history buffer.

Key SQL Macros used to create/stage records (These are referenced in the Getting Started portion of this document)	
Create_Schema_sqlite.sql	Creates the database schema that describes the database used by Musica. Internally called by the musica_db_sql_provision_sqlite.py script.
Create_Staging_sqlite.sql	An intermediate, mirror of the recordings table. Any changes created are created and 'Staged' prior to being promoted to the database.
Promote_Recordings_sqlite.sql	Promotes valid staging into production.
Audit_Recordings_sqlite.sql	performs read-only integrity and consistency checks on the recordings table, highlighting missing values, duplicates, and formatting inconsistencies without modifying data.
promote_staging_sqlite.sql	Promotes the staged records into the recordings and fills in NULL fields with the appropriate characters to present a clean and readable content display.
recordings_load_sql.sql	Loads the contents of a batch csv file into the Musica database. The

	contents are placed in the stg_recordings database while they await promotion.
v_recordings_display_sqlite.sql	This creates a view of the database for display within the DB program. It is a default display view.

Appendix C: List of Application and SQL sequences

SQL Queries and Display	
These provide a small sample of common queries/searches/etc. More help can be found at the SQLite website (@ https://www.sqlite.org/) or by searching online for other examples.	
Canonical library display (the “default”)	<pre>SELECT * FROM v_recordings_display ORDER BY artist, year, title;</pre>
All recordings by artist (chronological)	<pre>SELECT * FROM v_recordings_display WHERE artist = 'Davis, Miles' ORDER BY year, title;</pre>
Find duplicates (pre-promotion sanity check)	<pre>SELECT artist, title, year, COUNT(*) AS copies FROM recordings GROUP BY artist, title, year HAVING COUNT(*) > 1;</pre>
Check staging before promotion	<pre>SELECT * FROM stg_recordings WHERE is_valid = 1 ORDER BY artist, year, title;</pre>
What’s missing metadata	<pre>SELECT artist, title, year FROM recordings WHERE composer = '---' OR orchestra = '---' OR conductor = '---' ORDER BY artist, year;</pre>
Sort by Artist/Year and Title (Ascending)	<pre>SELECT * FROM v_recordings_display ORDER BY artist ASC, year ASC, title ASC;</pre>
QUERY a database (required from EDIT/Delete)	<pre>.headers on .mode box SELECT rowid AS id, artist, title, year,</pre>

	<pre> label, format, recording_mode, reissue FROM recordings WHERE <search_condition> ORDER BY artist, year, title; Example(s): A - Find a given title WHERE title LIKE '%Kind of Blue%' B - Find a artist WHERE title LIKE '%Kind of Blue%' C - Find a label WHERE label = 'Verve' </pre>
DELETE a specific record	<pre> BEGIN; DELETE FROM recordings WHERE rowid = <record_id>; SELECT * FROM recordings WHERE rowid = <record_id>; COMMIT; </pre>
EDIT a specific record - Only supported workflow NOTE – any field can be ‘modified’ (in this case, genre is being changed to CLASSICAL). Refer to the MUSICA Database Schema at the beginning of this document.	<pre> BEGIN; UPDATE recordings SET <column_name> = '<new_value>' WHERE rowid = <record_id>; SELECT * FROM recordings WHERE rowid = <record_id>; COMMIT; </pre>
EDIT a multiple columns - Only supported workflow as long as scope remains a single ID.	<pre> SELECT * FROM recordings WHERE id = 123; </pre>

	<pre> UPDATE recordings SET genre = 'CLASSICAL', orchestra = 'Berlin Philharmonic', recording_mode = 'S' WHERE id = 123; </pre>
v_recordings_display presentation view - Its sole purpose is to: • Normalize how data is displayed • Hide internal storage quirks • Provide a safe, consistent “read interface” for humans	<pre> CREATE OR REPLACE VIEW v_recordings_display AS SELECT artist, title, year, COALESCE(composer, '---') AS composer, COALESCE(orchestra, '---') AS orchestra, COALESCE(conductor, '---') AS conductor, genre, format, COALESCE(label, '---') AS label, COALESCE(catalog_number, '---') AS catalog_number, COALESCE(recording_mode, '-') AS recording_mode, COALESCE(reissue, '-') AS reissue, COALESCE(dbx_encoded, '-') AS dbx_encoded FROM recordings; </pre>
Export a (sub)set of records from the musica.db	<pre> .headers on .mode csv .separator ";" .output /home/<user>/Musica/data/exports/<export_filename>.csv SELECT artist, title, year, label, format, recording_mode, reissue FROM recordings WHERE <search_condition> ORDER BY artist, year, title; </pre>

	.output stdout
Application scripts - <user_home>/Musica/install	
musica_db_install.py	Installs the application architecture in the users home directory and confirms the integrity of the application tree.
Admin scripts - <user_home>/Musica/bin	
musica_db_clean_slate_sql.py	Reset Musica single-user SQLite test state. Options: --no-backup : skip backup --touch : recreate empty db file (0 bytes) after delete --wipe-logs : delete ~/Musica/logs/* (non-recursive by default) --wipe-imports : delete ~/Musica/data/imports/* (if present) --wipe-exports : delete ~/Musica/data/exports/* (if present)
musica_db_debug.py	Portable DB inspection tool for Musica (SQLite)
musica_db_sql_provision_sqlite.py	Musica SQLite provisioning (admin utility) Default (writes): - Ensure bootstrap objects exist (musica_meta, musica_users, v_musica_users, trigger) - Ensure current OS user exists in musica_users with role='owner' - Load SQLite install schema from ~/Musica/sql/install/*.sql (tables/views/triggers/indexes) - Stamp schema_version + provisioned_by
Support scripts - <user_home>/Musica/bin	
musica_db_add_recording_sqlite.py	Interactive single-record insert (SQLite) Rules: - Always prompt: artist, title, year, genre, format - Prompt composer/orchestra/conductor ONLY when genre == 'Symphonic'
musica_db_batch_load_sqlite.py	% musica_db_batch_load_sqlite.py <import_batch>.csv The command searches for the named file from the Musica/data/imports directory.
SQL install - <user_home>/Musica/sql/install	
Audit_Recordings_sqlite.sql	Audit lookup table when using record id index
Create_Schema_sqlite.sql	SQL instruction that describes the Musica database schema
Create_Staging_sqlite.sql	SQL sequence that creates staging view

v_recordings_display_sqlite.sql	SQL that creates read-only display view of entire database
SQL runtime - <user_home>/Musica/sql/runtime	
Promote_Recordings_sqlite.sql	Promote valid staging records into production (SQLite)
promote_staging_sqlite.sql	Promote valid staging rows to recordings (SQLite)
recordings_load_sqlite.py	Load a CSV file into stg_recordings (SQLite). Target in <user_home>/Musica/data/import/batch_load.csv